DREW

# Lecture 3: Data Representation

CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

08/30/2024

# §2.5: Operations on 2's Comp Binary

- **Recall**: a data type includes both **representation** and **operations**.
- We now have a good representation for *signed integer*; i.e., Two's Complement Binary Representation.
  - To change sign in **2's Comp.**: Flip the bits and plus one.
- Base 2 Addition: just like in base 10.

```
  10010        10010        1111
+  1001      +  1011      +     1
-------      -------      -------
  11011        11101        10000
```

carry

carry

DREW

# Addition

- 2's comp. addition is simply binary addition.
- Assume operands have the same bit length; ignore carry-out for now.
- Assuming 8-bit 2's complement numbers:

$$\begin{array}{rl} 01101000 & (104)_{10} \\ +\ 11110000 & (-16)_{10} \\ \hline 01011000 & (88)_{10} \end{array}$$

$$= 8 + 16 + 64 = 88$$

$$\begin{array}{rl} 11110110 & (-10)_{10} \\ +\ 11110111 & (-9)_{10} \\ \hline 11101101 & (-19)_{10} \end{array}$$

$$= -(00010011)_2$$

$$= -(1 + 2 + 16) = -19$$

DREW

# Subtraction

- **Recall**: In base 10, $x - y$ is defined as $x + (-y)$.
- Similarly, subtraction is addition with the 2nd operand negated.
- Assume operands have the same bit length; ignore carry-out for now.
- Assuming 8-bit 2's complement numbers:

$$\begin{array}{rl} 01101000 & (104)_{10} \\ -\ 00010000 & (16)_{10} \\ \hline 01101000 & (104)_{10} \\ +\ 11110000 & (-16)_{10} \\ \hline 01011000 & (88)_{10} \end{array} \qquad \begin{array}{rl} 11110110 & (-10)_{10} \\ -\ 00001001 & (9)_{10} \\ \hline 11110110 & (-10)_{10} \\ +\ 11110111 & (-9)_{10} \\ \hline 11101101 & (-19)_{10} \end{array}$$

DREW

# Sign Extension

- To add two numbers, we must represent them with the same number of bits.
- In 2's comp., to extend the length while keeping the sign and value the same, we **replicate the m.s.b** until the operand with less bits has the same bit length as the other operand.
- As in 2's comp., leading zeros do not affect positive numbers and zero, and leading ones do not affect negative numbers.

| | 4-bit | 8-bit |
|---|---|---|
| $(4)_{10}$ | 0100 | 00000100 |
| $(-4)_{10}$ | 1100 | 11111100 |

# Overflow

- **Overflow**: If the result of the operation exceeds the range that can be represented by the number of bits.

- **Recall**, a $k$-bit 2's comp. bin. number ranges from $-2^{k-1} \sim 2^{k-1} - 1$.
- Hence, 5-bit ranges from $-16 \sim 15$.

$$\begin{array}{rl} 01000 & (8)_{10} \\ +\ 01001 & (9)_{10} \\ \hline 10001 & (-15)_{10} \end{array} \qquad \begin{array}{rl} 11000 & (-8)_{10} \\ +\ 10111 & (-9)_{10} \\ \hline 01111 & (+15)_{10} \end{array}$$

# Overflow

$$\begin{array}{rrl} & 01000 & (8)_{10} \\ + & 01001 & (9)_{10} \\ \hline & 10001 & (-15)_{10} \end{array} \qquad \begin{array}{rrl} & 11000 & (-8)_{10} \\ + & 10111 & (-9)_{10} \\ \hline & 01111 & (+15)_{10} \end{array}$$

- How do we detect overflow?
  - **Q**: Can the sum of two positive (or zero) integers be negative?
    Can the sum of two negative integers be positive?

- **One Test**: There is an overflow if the signs of both operands are the same, but the sign of the sum is different.
- **Another test**: the "carry in" to the m.s.b is different from the "carry out" from the m.s.b..
- This is easier for hardware to detect, as the ALU has monitors the "carry out" from + and shifting, and the borrowing from -.

# Overflow

In Java, cast int to long.

- How to deal with overflow? Need more bits → Sign extension

$$\begin{array}{rll} & 01000 & (8)_{10} \\ + & 01001 & (9)_{10} \\ \hline & 10001 & (-15)_{10} \end{array}$$

$$\begin{array}{rll} & 11000 & (-8)_{10} \\ + & 10111 & (-9)_{10} \\ \hline & 01111 & (+15)_{10} \end{array}$$

$$\begin{array}{rll} & 001000 & (8)_{10} \\ + & 001001 & (9)_{10} \\ \hline & 010001 & (17)_{10} \end{array}$$

$$\begin{array}{rll} & 111000 & (-8)_{10} \\ + & 110111 & (-9)_{10} \\ \hline & 101111 & (-17)_{10} \end{array}$$

# Shifting and Multiplication

- What if we add a number to itself; i.e., $x + x$ or $2x$?
- Sure, we can simply see it as just an addition.

- However, recall the exponent formula:

$$x = d_{k-1} \cdot 2^{k-1} + \cdots + d_1 \cdot 2^1 + d_0 \cdot 2^0,$$

then, adding the number to itself, or doubling it, becomes,

$$x + x = (d_{k-1} \cdot 2^{k-1} + \cdots + d_0 \cdot 2^0) + (d_{k-1} \cdot 2^{k-1} + \cdots + d_0 \cdot 2^0)$$
$$= 2x = 2\left(d_{k-1} \cdot 2^{k-1} + \cdots + d_0 \cdot 2^0\right)$$
$$= d_{k-1} \cdot 2^k + \cdots + d_1 \cdot 2^2 + d_0 \cdot 2^1 + 0 \cdot 2^0$$

Now, let's reconstruct its binary ... $d_{k-1}$ $d_{k-2}$ $d_1$ $d_0$ $0$

# Shifting and Multiplication

- **Ex.**: $(59)_{10} + (59)_{10} = (118)_{10}$, and in binary $(111011)_2 + (111011)_2$

$$= (1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) + (\dots)$$
$$= 2 \cdot (0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)$$

$$= 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$$
$$= 2 + 4 + 16 + 32 + 64 = 118$$

- Note that, this is a *Shift* to the left by 1 place:

| | |
|---|---|
| before doubling | 111011 |
| after doubling | 1110110 |

DREW

# Shifting and Multiplication

- In binary representation, multiplying a number by a power of 2 is the *Left Shift* operation.

- Multiplying by 2 → Shift to the left by 1 place (padding a 0 in the l.s.b)
- Multiplying by 4 → Shift to the left by 2 places (padding two 0's in l.s.b)
- …
- Multiplying by $2^k$ → Shift to the left by $k$ places (padding $k$ 0's in l.s.b)

- **Note**: Given fixed bit length, this may create overflows!
- Conversely, division is the *Right Shift* operation.

# Shifting and Multiplication

- Now, what if a binary number is multiplied by another number that is NOT a power of 2?
- **Ex.**: $(4)_{10} \times (5)_{10} = (??)_2$

Note that multiplication is commutative.
Shift the bin. rep. of 5 instead.

- What if both are not powers of 2?
- **Ex.**: $(11)_{10} \times (7)_{10} = (??)_2$

$= (11)_{10} \times (1 + 2 + 4)_{10}$

… …

Split one num into parts that are powers of 2.
Then, distribute the multi.
Shiftings with additions.

# Bitwise Logical Operations

| $p$ | $\neg p$ |
|---|---|
| 1 | 0 |
| 0 | 1 |

Logical NOT/
Complement, ¬

| $p$ | $q$ | $p \wedge q$ |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |

Logical AND/
Conjunction, ∧

| $p$ | $q$ | $p \vee q$ |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 0 | 0 |

Logical OR/
Disjunction, ∨

- View an $n$-bit binary number as a sequence of $n$ individual bits:
  - Operation applied to each corresponding pair of bits.

# Bitwise Logical Operations

- **Ex.**: $a = (11000101)_2$ and $b = (00001111)_2$, then

- AND, OR are binary op.s
  - Take 2 operands.

$$\begin{array}{rr} & 11000101 \\ AND & 00001111 \\ \hline & 00000101 \end{array} \qquad \begin{array}{rr} & 11000101 \\ OR & 00001111 \\ \hline & 11001111 \end{array}$$

- NOT is a unary op.
  - Takes 1 operand.

$$\begin{array}{rr} NOT & 11000101 \\ \hline & 00111010 \end{array}$$

# Bitwise Logical Operations

- *Precedence* of Logical Connectives

  [From left to right]

  1. Parentheses — *Highest*
  2. NOT *(as if there was parentheses)*
  3. AND
  4. OR — *Lower*

  [**However**, good practice is to always be using parentheses.]

$0010\ AND\ NOT(0101\ OR\ 1101)$

$= 0010\ AND\ NOT(1101)$

$0010\ AND\ 0010$

$0010$

# Bitwise Logical Operations

- Bitwise XOR (Exclusive OR), denoted with $\oplus$
  - TRUE = 1
  - FALSE = 0

| $p$ | $q$ | $p \oplus q$ |
|---|---|---|
| 1 | 1 | 0 |
| 1 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 0 | 0 |

# Properties of Bitwise Logical Operations

- All the properties/laws/identities that apply to logical operations still function in bitwise logic; e.g., commutativity.

- **De Morgan's Law**:
  - $\neg(A \wedge B) = \neg A \vee \neg B$
  - $\neg(A \vee B) = \neg A \wedge \neg B$

| $A$ | $B$ | $A \wedge B$ | $\neg(A \wedge B)$ |
|---|---|---|---|
| 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 0 | 0 | 1 |

| $A$ | $B$ | $\neg A$ | $\neg B$ | $\neg A \vee \neg B$ |
|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 1 |

# Properties of Bitwise Logical Operations

- **De Morgan's Law**:
  - $\neg(A \wedge B) = \neg A \vee \neg B$
  - $\neg(A \vee B) = \neg A \wedge \neg B$

- In fact, given either AND, NOT, or OR, NOT, we can create the third operation and any other logical operation by applying De Morgan's Law.

- Important concept for digital circuit design.
- Will re-visit next week.

# Number with Fractions: Fixed-Point

- **Fixed-Point Fractions**: Use a "binary point" to separate positive exponents from negative exponents:
  - $(101000.101)_2 = 2^5 + 2^3 + 2^{-1} + 2^{-3} = 40.625$
  - Can represent a number quite precisely.

- 2's Complement addition, subtraction, multiplication still works:
  - Just need to align the binary points.

$$\begin{array}{rl} 00101000.101 & (40.625)_{10} \\ +\ 11111110.110 & (-1.25)_{10} \\ \hline 00100111.011 & (39.375)_{10} \end{array}$$

# Number with Fractions: Floating Point

- However, if we need to represent an extremely large or extremely small number, we need a better representation.
  - *Avogadro's Number*: $6.023 \times 10^{23}$ requires 79 bits
  - *Plank's Constant*: $6.626 \times 10^{-34}$ requires > 110 bits
  - In fact, other than the exp., only 4 digits are significant to us!
- Instead of setting all bits to express *precision*, some will be used for *range*.
- To increase the *range*, use an equivalence to the "scientific notation".
  - $sign\ \ fraction \times 2^{E}$

# Number with Fractions: Floating Point




- *32-bit Floating-Points*
  - 1 bit for the sign (positive or negative);
  - 8 bits for range (exponent);
  - 23 bits for precision (fraction), same as Fixed-Point representation.
  - **Normalized Form**: the most common form
    when $1 \leq exp \leq 254$, $N = (-1)^S \times 1.frac \times 2^{exp-127}$
- **Ex.**: 10111111010000000000000000000000 in 32-bit Float-Point has the value $= (-1)^1 \times 1.5 \times 2^{126-127} = -1.5 \times 2^{-1} = -0.75$.

# Number with Fractions: Floating Point

- **Normalized Form**: the most common form
  when $1 \leq exp \leq 254$, $N = (-1)^S \times 1.frac \times 2^{exp-127}$

- **Ex.**: Express $\left(-6\frac{5}{8}\right)_{10}$ in 32-bit Floating-Point

$$= -(4 + 2 + 0.5 + 0.125)_{10} = (-1)^1 \times (110.101)_2$$

$$= (-1)^1 \times (1.10101) \times 2^2 \text{ [moving the binary point]}$$

Note: exp -127 = 2 → exp = 129

Hence, $\left(-6\frac{5}{8}\right)_{10} = (11000000110101000000000000000000)_2$ in 32-bit Floating-Point.

# Number with Fractions: Floating Point

**Special Forms**

- **Infinity**: positive 1111111110 … 0 and negative 0111111110 … 0
- **Subnormal Form**: for even smaller numbers

$$\text{when } exp = 0,\ N = (-1)^S \times 0.frac \times 2^{126}$$

- *32-bit Floating-Points* [IEEE 754]: most commonly used in computers nowadays
  - The *float* datatype in Java.
  - Also, there's *double (precision)* in Java, the *64-bit Floating Point*.

# Questions?

- **Next Lecture**:
  - The Text Data Type
  - Software overview for this course
  - In-Class Exercise with Attendance